User Input in C

Methods for Data Acquisition at Runtime

scanf() ▪ getchar() ▪ fgets() ▪ Format Specifiers

Overview

User input allows C programs to become interactive. Instead of hardcoding values, we allow the end-user to provide data while the program is running.

Key Concept: Input functions read data from the Standard Input (stdin), which is typically your keyboard.

The Address Operator (&)

One of the most critical parts of input in C is the & symbol. It tells `scanf()` the memory address where the data should be stored.

1. The scanf() Function

The most versatile and common function for taking formatted input.

```
int age;
printf("Enter age: ");
scanf("%d", &age);
```

How it works:

- **"%d"**: Tells the program to expect an integer.
- **&age**: Points to the specific box in memory named 'age'.

2. The getchar() Function

Specifically used when you only need to read a **single character**. Unlike `scanf`, it returns the character directly.

```
char grade;
printf("Enter your grade: ");
grade = getchar();
```

This is often used for simple menu choices (Y/N) or pressing any key to continue.

3. Reading Strings

While `gets()` exists in old textbooks, it is **dangerous** because it doesn't limit how much text is entered, often crashing programs.

The Modern Choice: fgets()

```
char name[30];
printf("Enter your full name: ");
fgets(name, sizeof(name), stdin);
```

This ensures the program only reads as much text as the array can safely hold.

Important Rules for Input

Rule	Explanation
Data Type Match	If you use %d for a char, the program will store garbage values.
No & for Arrays	When using `scanf` for strings, the & is not used because the array name is already an address.
Buffer Issues	Input stays in a "buffer." Sometimes from a previous entry affects the next one.

Module Complete

You have learned how to capture integers, characters, and strings safely in C.

*** Final Page ***